GoNature — Defense Navigation Guide

"If they ask about X, go to file Y" · keywords, Q&A, and tips · Group 9

The Golden Rule (memorize this one flow)

> **Every feature travels the same path.** When they ask "how did you do X," trace these 5 stops in order. If you only remember one thing: `EchoServer.java` **is the hub — every feature has a** `case` **there.** To find any feature, open EchoServer and Ctrl+F the command name.

```
1. CONTROLLER (gui/*Controller.java)  -> button handler builds a Message, calls ClientUI.chat.accept(msg)
2. ChatClient.java                     -> sends it, blocks waiting for reply (handleMessageFromClientUI)
3. EchoServer.java   <= THE HUB         -> switch-case on message type, calls a DBController method
4. DBController.java                    -> the actual SQL (PreparedStatement)
5. back up the chain                   -> ChatClient stores reply in a "lastXxx" field, controller reads it
```

Your default answer to ANY feature question: "The button handler in `XController` builds a `Message`, `ChatClient` sends it, `EchoServer` routes it in its switch, and `DBController` runs the SQL. The reply comes back through `ChatClient`'s static field."

Feature → Where-To-Look Map

🎫 Waiting List

Keywords to say	"promotion," "FIFO queue," "1-hour confirmation window," `promoteFromWaitingList`, idempotent
Client	`BookingController` (offers waitlist when full) → `WaitingListController` (employee view)
Server hub	`EchoServer` → cases `JOIN_WAITING_LIST`, `GET_WAITING_LIST`, `CONFIRM_VISIT`
Core logic	`DBController.promoteFromWaitingList()` **(line 837)** — triggered from `cancelOrder` (817), no-show sweep (1066), expiry (1190)
The timer	`WaitlistTimerService.java` — `schedulePromotionTimeout()` arms a 1-hour one-shot + a 5-min safety sweep
One-liner	"When a slot frees up (cancel or no-show), `promoteFromWaitingList` grabs the next person in line, creates a held order, and `WaitlistTimerService` gives them 1 hour to confirm — enforced two ways so a server restart can't lose it."

🗓️ Book a Visit

Keywords	`BookingRequest`, `checkAndBook`, "two-layer concurrency," price preview
Client	`BookingController.java` (form, live price, client validation) → `BookingConfirmationController` (result + QR pass)
Server hub	`EchoServer` → case `BOOK_ORDER` **(line 164)**
Core logic	`DBController.checkAndBook()` — the `SELECT … FOR UPDATE` is at **line 329**
One-liner	"Client validates first for fast feedback, server re-validates in `validateBookingRequest`, then `checkAndBook` inserts the order inside a locked transaction so the last slot can't be double-booked."

🔒 Synchronization / Concurrency *(they love this one)*

Keywords	"two independent layers," JVM lock + DB lock, `parkLocks`, `SELECT … FOR UPDATE`, race condition
Layer 1 (app)	`EchoServer.java` line **83** (`parkLocks` map) and line **184** (`synchronized (parkLock)`) — serializes booking attempts *per park*
Layer 2 (DB)	`DBController.checkAndBook()` line **329** — `SELECT … FROM parks WHERE id=? FOR UPDATE` locks the row in the transaction
Other synced	`promoteFromWaitingList` (837), `expireWaitlistEntry` (1170), `confirmPromotedVisit` (1202); all singletons use `synchronized getInstance()`
One-liner	"Two visitors clicking 'book' on the last spot at the same instant: the JVM `synchronized(parkLock)` lets only one thread into `checkAndBook` at a time, and even across multiple servers the DB's `FOR UPDATE` row lock is the backstop. One succeeds, the other gets 'full'."

📊 Reports / Graphs

Keywords	`BarChart` / `LineChart` (JavaFX), `XYChart.Series`, interval-overlap occupancy
Client	`ReportsController.java` — `loadVisitorReport()` (280), `loadCancelReport()` (340), `loadCancelDistribution()` (384), `loadUsageReport()` (425), `loadDurationReport()` (490)
Server hub	`EchoServer` → `GET_VISITOR_REPORT`, `GET_CANCEL_REPORT`, `GET_USAGE_REPORT`, etc.
Core SQL	`DBController.getUsageReport()` (2084 — interval-overlap query), `getVisitorReport()`, `getCancellationDistribution()`
Usage gotcha	If asked "why is the usage % low": it measures **concurrent occupancy** (present if `entry ≤ hour < exit`), not arrivals. That's the interval-overlap join.

🌙 Dark Mode

Keywords	"single static flag," `dark-mode` CSS class, watches `Window.getWindows()`
Logic	`ThemeManager.java` — `toggle()` (82), `installToggle(Button)` (113), constant `DARK_CLASS = "dark-mode"` (34)
The colors	`gonature.css` → all the `.dark-mode` rules
One-liner	"One static flag. `ThemeManager` adds/removes the `dark-mode` style class on every open window's root, and the `.dark-mode` CSS rules do the recoloring — so every current and future screen follows automatically."

🔑 Login / Single-Session

Keywords	auto-detect role, `activeSessions` set, "already logged in elsewhere"
Client	`LoginController.java` (one form, no role dropdown)
Server hub	`EchoServer` → `LOGIN_VISITOR` (111), `LOGIN_USER` (130)
Core logic	`DBController.loginVisitor()` / `loginUser()`; single-login enforced by `activeSessions.add()` returning false (EchoServer 116, 143)

🖂 Email Notifications

Keywords	Gmail SMTP, SSL port 465, daemon thread pool, "never blocks the request"
Logic	`EmailService.java` — `sendEmail()` on a bounded thread-pool; credentials from `db.properties`
Called from	`EchoServer` (booking confirm, cancel, register) and the three background services

⏰ Daily Reminders / No-Show *(the "runs forever" story)*

Keywords	`ScheduledExecutorService`, `scheduleAtFixedRate`, daemon thread, "no restart needed"
Reminders	`ReminderService.java` — daily at 00:05 via `secondsUntilNextDailyRun()`
No-show	`NoShowService.java` — sweeps every 5 min, 15-min grace
Armed at	`EchoServer.serverStarted()` lines 1112-1114

🚪 Check-In / Exit (Gate)

Keywords	pre-booked vs walk-in, `EntryCheckRequest`, occupancy
Client	`EntryControlController.java` (two tabs)
Server hub	`EchoServer` → `FIND_TODAY_BOOKING`, `CHECK_IN_VISITOR`, `REGISTER_EXIT`
Core logic	`DBController.checkInVisitor()` (1506), `registerExit()` (1837)

💸 Promotions & Settings Approval (two-step workflow)

Keywords	PENDING → ACTIVE, park manager proposes / dept manager approves
Client	`PromotionsController` + `ParkSettingsController` (propose) → dept manager approves in their panel
Server hub	`EchoServer` → `SUBMIT_PROMOTION`, `APPROVE_PROMOTION`, `SUBMIT_PARK_SETTINGS`, `APPROVE_SETTINGS`
Pricing uses it	`DBController.getBestActivePromotion()` (3032)

Likely Q&A

Q: Walk me through what happens when a visitor books a visit.

> The `BookingController` validates the form client-side, wraps it in a `BookingRequest` inside a `Message`, and `ChatClient` sends it and blocks. `EchoServer`'s `BOOK_ORDER` case re-validates, acquires the per-park lock, and calls `checkAndBook` which runs `SELECT … FOR UPDATE` and inserts the order. The server sends back a `BookingResult` with a price preview, fires a confirmation email, and the client shows the confirmation screen with a QR pass.

Q: How do you prevent two people booking the last spot at the same time?

> Two independent layers. In `EchoServer`, a `synchronized(parkLock)` block (one lock object per park) lets only one booking thread per park run at a time. Inside the DB transaction, `checkAndBook` does `SELECT … FOR UPDATE` on the park row, which is the backstop even with multiple servers. (Point to EchoServer:184 and DBController:329.)

Q: Does the server need restarting for daily reminders?

> No. `ReminderService` uses a `ScheduledExecutorService.scheduleAtFixedRate` aligned to 00:05 every day. It runs once at startup, then every 24h. A try/catch wraps each run so a transient DB error can't kill the schedule.

Q: How does the client get updates — does the server push?

> No push. It's **pull-based**: the client sends a request and blocks on `awaitResponse` until the reply lands in a static field in `ChatClient` (15-sec safety timeout). On multi-client sync: clients refresh when they re-open a screen; correctness is guaranteed at the DB level by the locks, not by live UI sync.

Q: Why is there a separate `OrderDetail` and `Order`?

> `Order` is a small write/edit payload (`UPDATE_ORDER`); `OrderDetail` is the full read-only row for display. Separating read and write models keeps each lean.

Q: What's the waiting-list promotion flow exactly?

> When a slot frees (cancel, no-show, or another expiry), `promoteFromWaitingList` picks the next queued visitor, creates a held PENDING order, marks them NOTIFIED with a 1-hour deadline, and `WaitlistTimerService` schedules expiry. If they don't confirm within the hour, `expireWaitlistEntry` (idempotent) releases it and promotes the next person.

Q: Where's your database code — is it spread around?

> No, it's **all** in `DBController.java`, one singleton, every query a parameterized `PreparedStatement`. Controllers never touch SQL — they only send `Message`s.

Q: How is the database password kept secure?

> The MySQL password is never stored. `db.properties` only holds the URL and username; the password is typed into the server's startup dialog at runtime (`ServerUI`).

Q: What design patterns did you use?

> **Singleton** (`DBController` , `UserSession` , the services), **Facade** (`ClientController` over `ChatClient`), **MVC** (FXML view + Controller + DTO model), and the **Command** style dispatch in `EchoServer` 's switch.

Tips & Tricks for the Defense

1. Always start at the controller, end at DBController. Examiners want to see you understand the *layers*, not just one file.

2. `EchoServer` **is your index.** If you blank, say "let me open EchoServer and find the case" — every feature is there, and the case name tells you the DBController method.

3. Name the file out loud. "That's in `WaitlistTimerService` " sounds far more confident than "somewhere in the server."

4. For concurrency, always say "two independent layers." That phrase shows you understood it's not just one `synchronized` .

5. Know your three background threads cold (Reminder / Waitlist / No-show) — examiners love "what happens if the server runs for a week?"

6. If asked something you don't know, fall back to the golden-rule flow and trace it live — the structure is so consistent you can reason it out.

7. Distinguish "real" vs "simulated": emails are **real** (Gmail SMTP); QR codes and the notification inbox are **simulated**. Be honest — it's a common trap question.